



AMBA

APB Peripheral

I AI 시스템 반도체 설계 2기

I 엄 찬 하



대한상공회의소

guichanha00@gmail.com

목차 페이지

01

프로젝트 개요

02

AMBA

03

Peripheral

04

Simulation & Verification

05

Troubleshooting

06

고찰



01

프로젝트 개요

프로젝트 개요

목표

1. RISC-V Multi Cycle CPU와 AMBA APB Protocol을 이용한 Peripheral 설계
2. 클래스 기반 사용자 정의 테스트 벤치를 통한 DUT 검증
3. C언어를 통한 시스템 기능 구현

프로젝트 개요

개발 환경



Tool

VIVADO 2020.2



FPGA

Basys3



Language

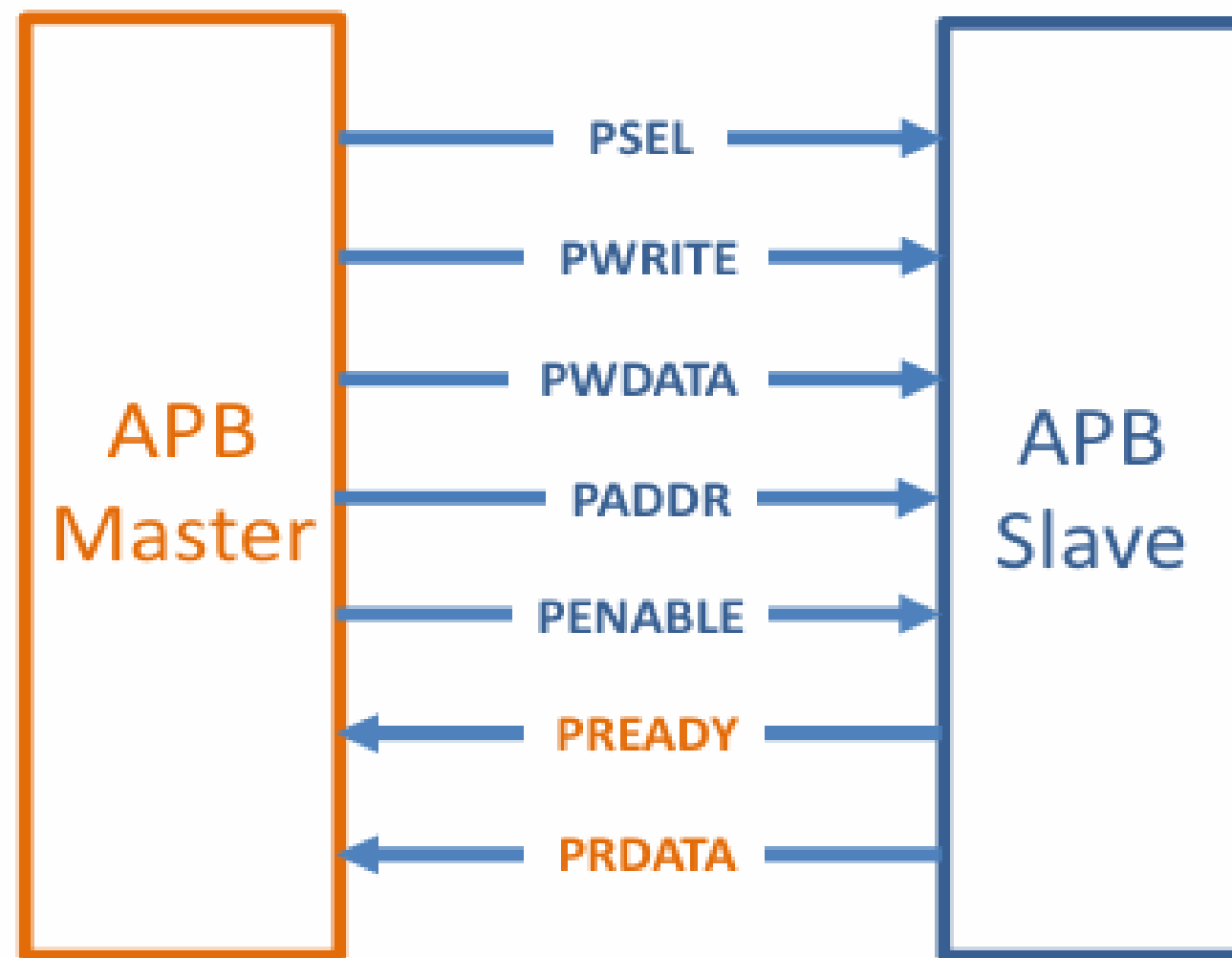
System Verilog / C 언어



02

AMBA APB

AMBA APB



APB란 무엇인가?

ARM에서 정의한 AMBA 계열의 버스 프로토콜 중 하나로,
주변 장치와의 저속 통신을 담당

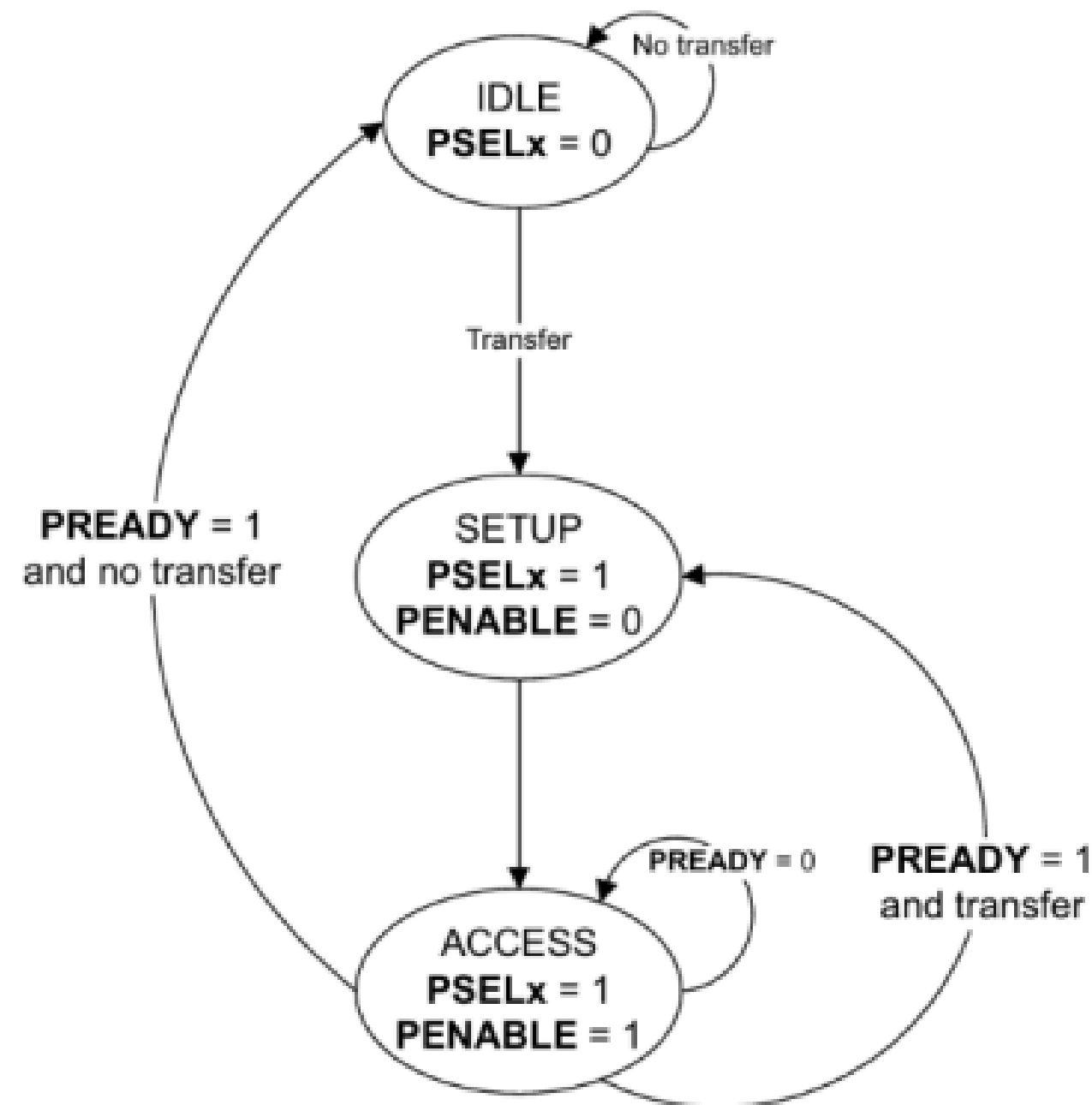
장점

- 구현이 매우 단순
- 저전력 소비 (모바일 , 임베디드 시스템에 적합)
- 예측 가능한 동작 타이밍 (클럭 기반 2단계 구조)

특징

- 주소 기반 통신 (Memory mapping)
- 단일 Master + 다수 Slave 구조로 제한적

AMBA APB



클럭 기반 2단계 프로토콜

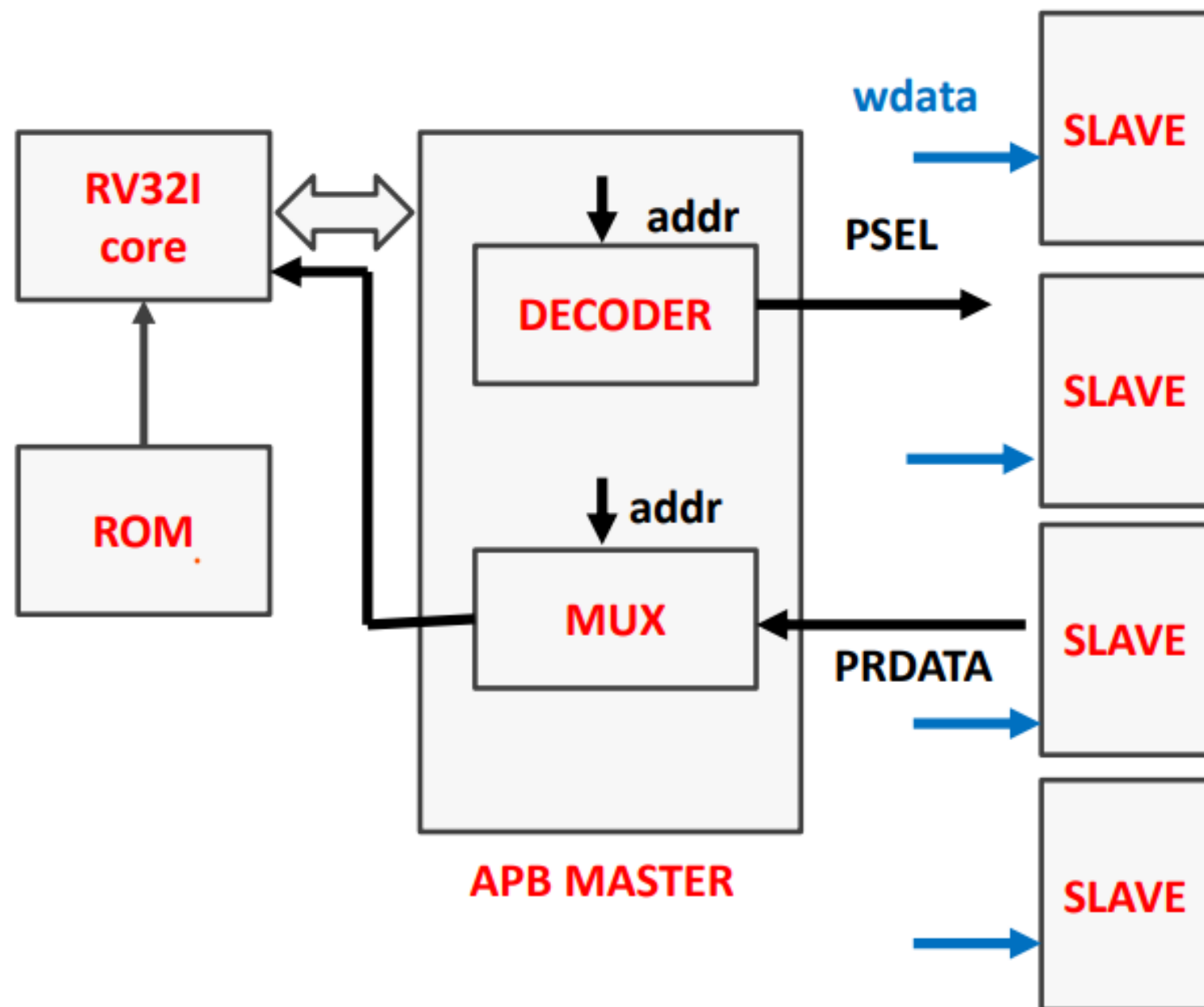
Setup 단계

- 주소, 읽기/쓰기 신호, 쓰기 데이터 등이 설정만 되는 단계
- 실제로 데이터 전송은 일어나지 않음

Access 단계

- 슬레이브가 신호를 읽고 응답(PREADY = 1시 전송 완료)
- Write이면 데이터가 슬레이브에 기록되고, Read라면 데이터가 PRDATA로 반환됨

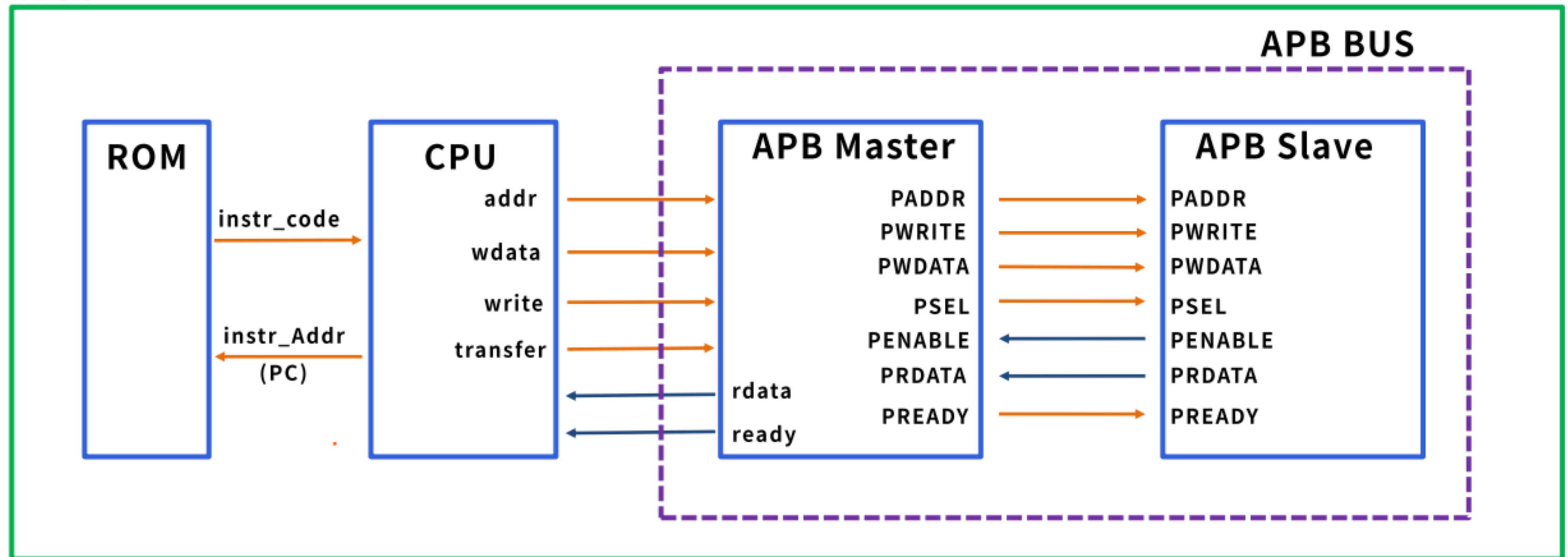
AMBA APB



구조도 - Address

- CPU 에서 Broad Casting
- Addr를 기준 Decoder -> Chip Enable

AMBA APB

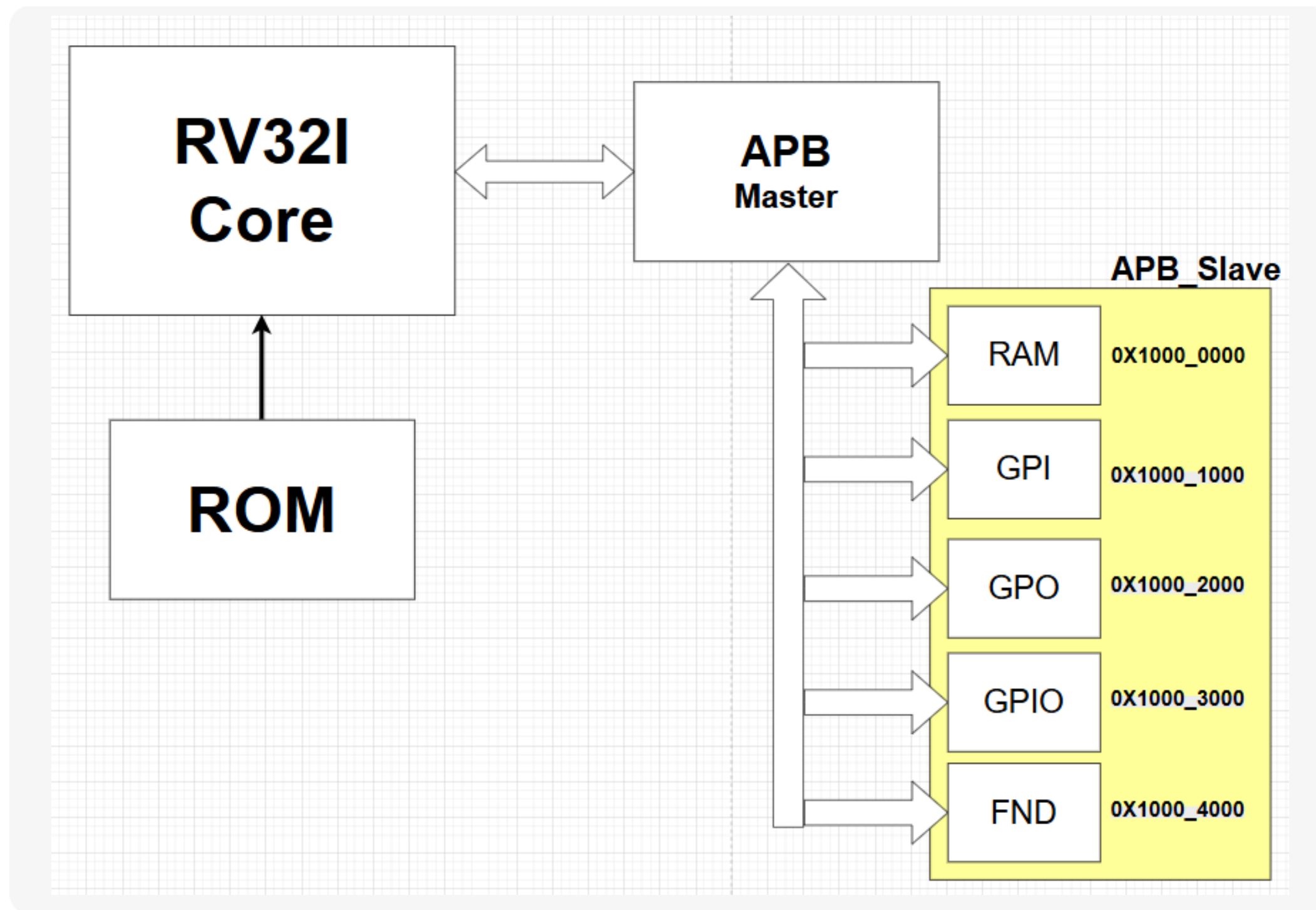
MCU



03

Peripheral

Peripheral

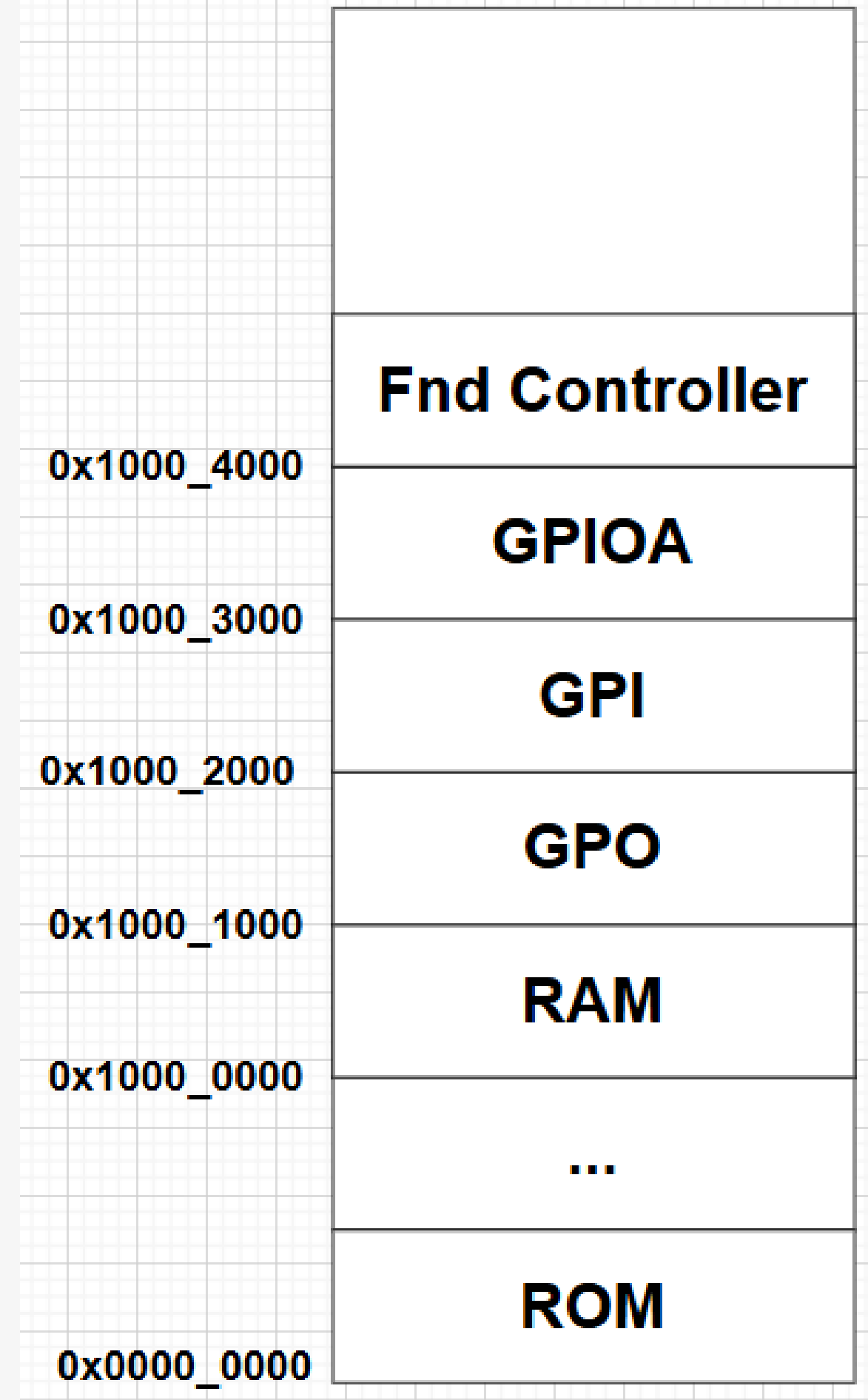


Addr $\leftrightarrow$ Memory

해당 주소 값에 따라 Peripheral 한
모듈을 연결

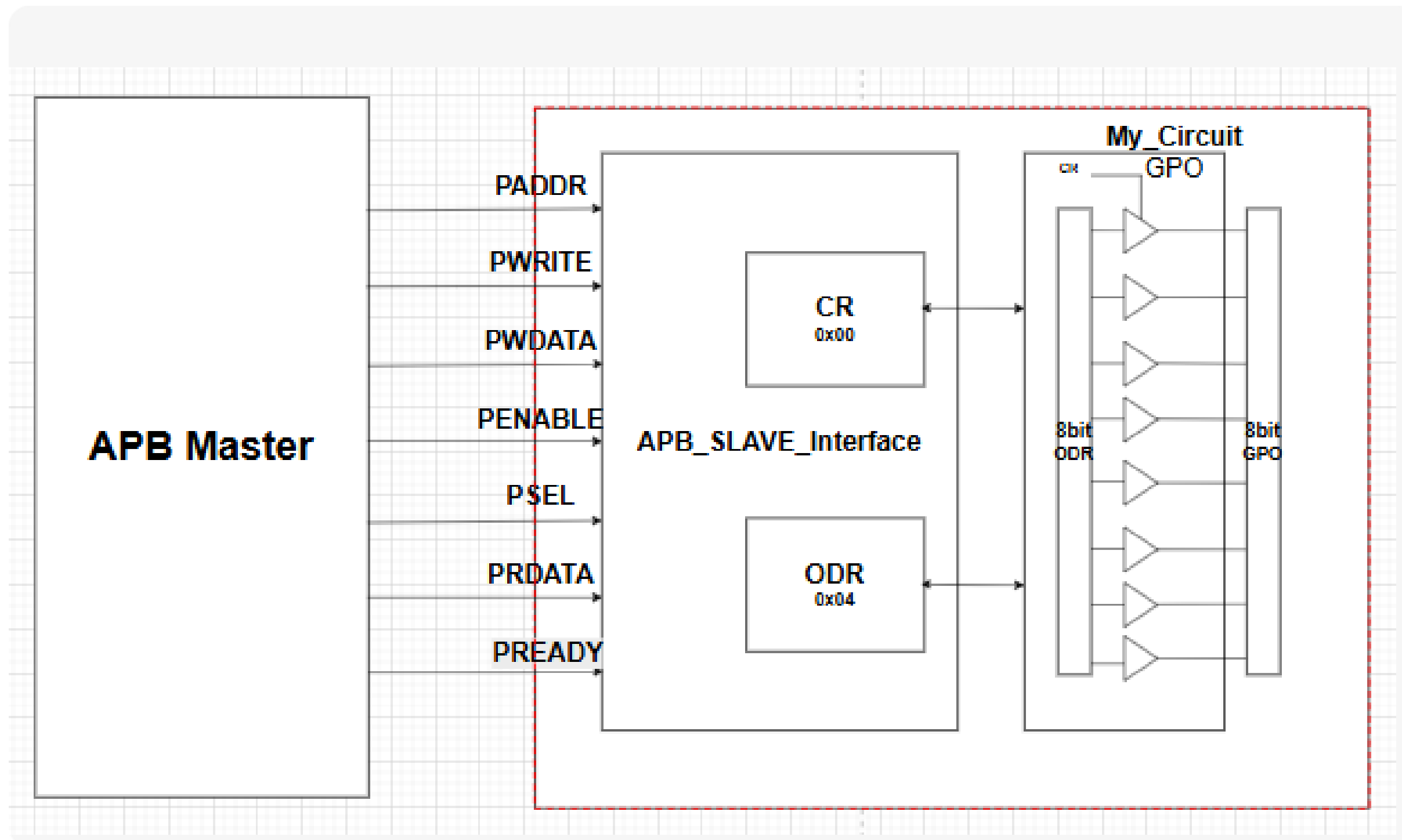


Peripheral



Memory Map

Peripheral

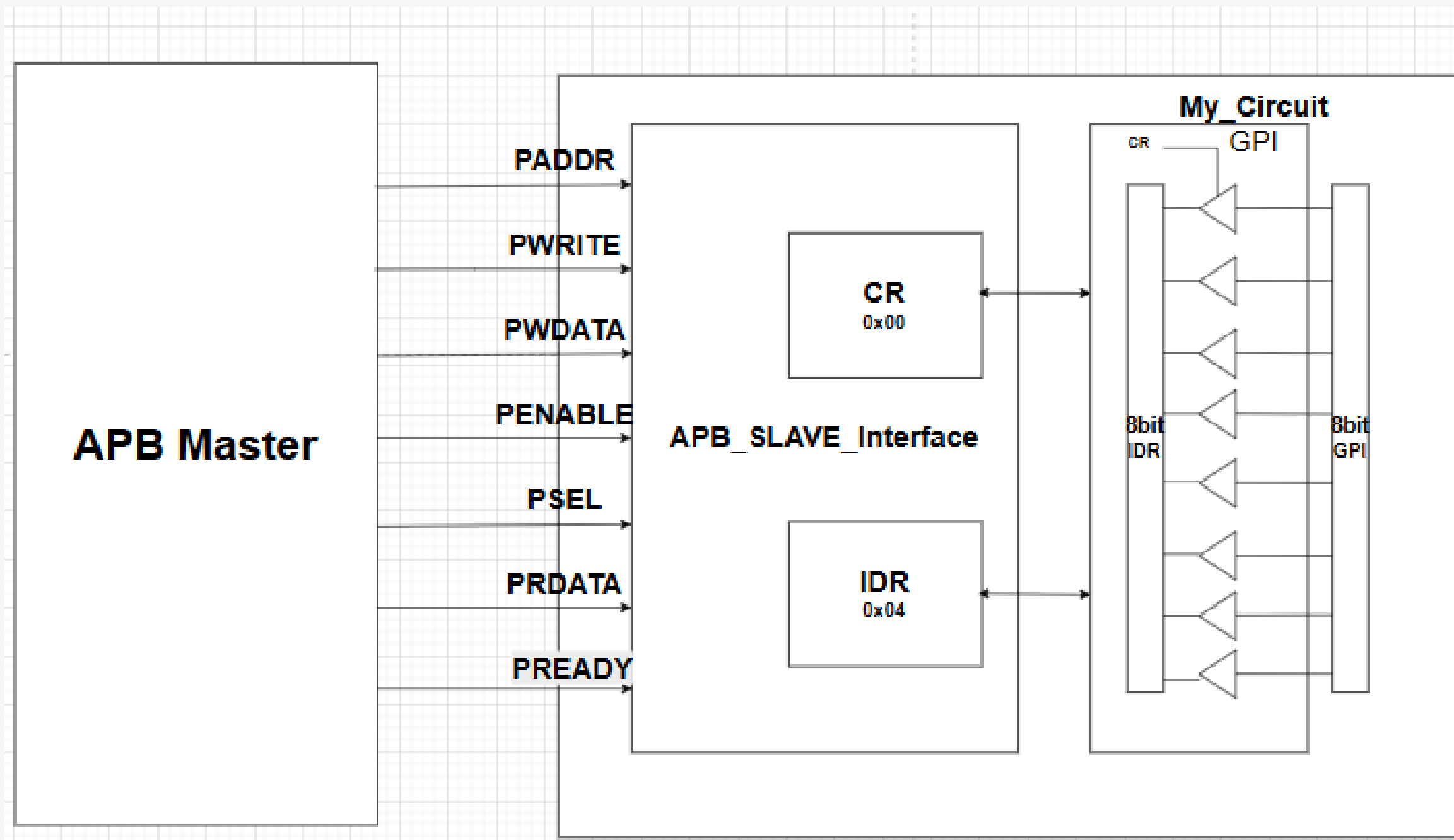


GPO

Control Register

Output Data Register

Peripheral

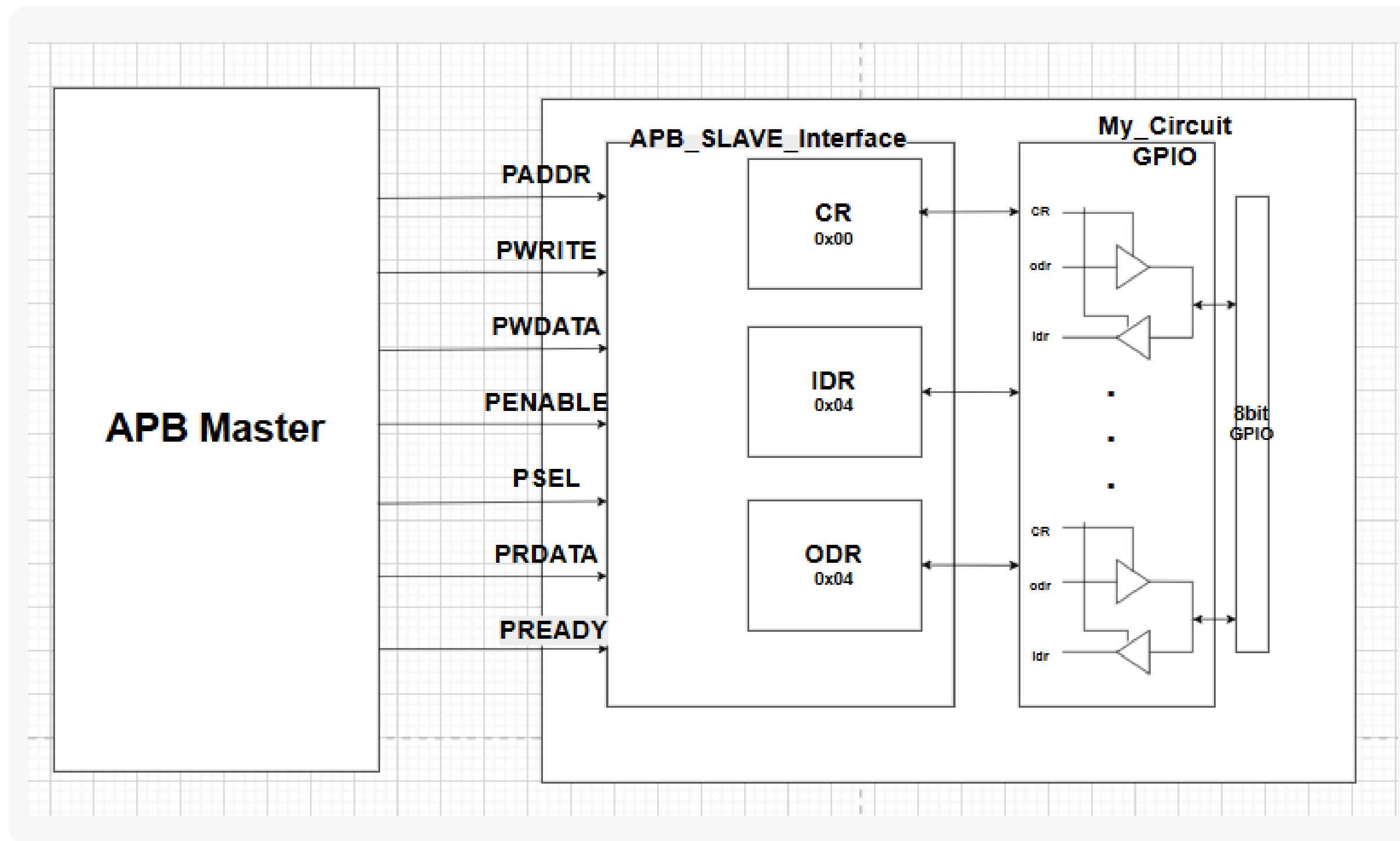


GPI

Control Register

Input Data Register

Peripheral



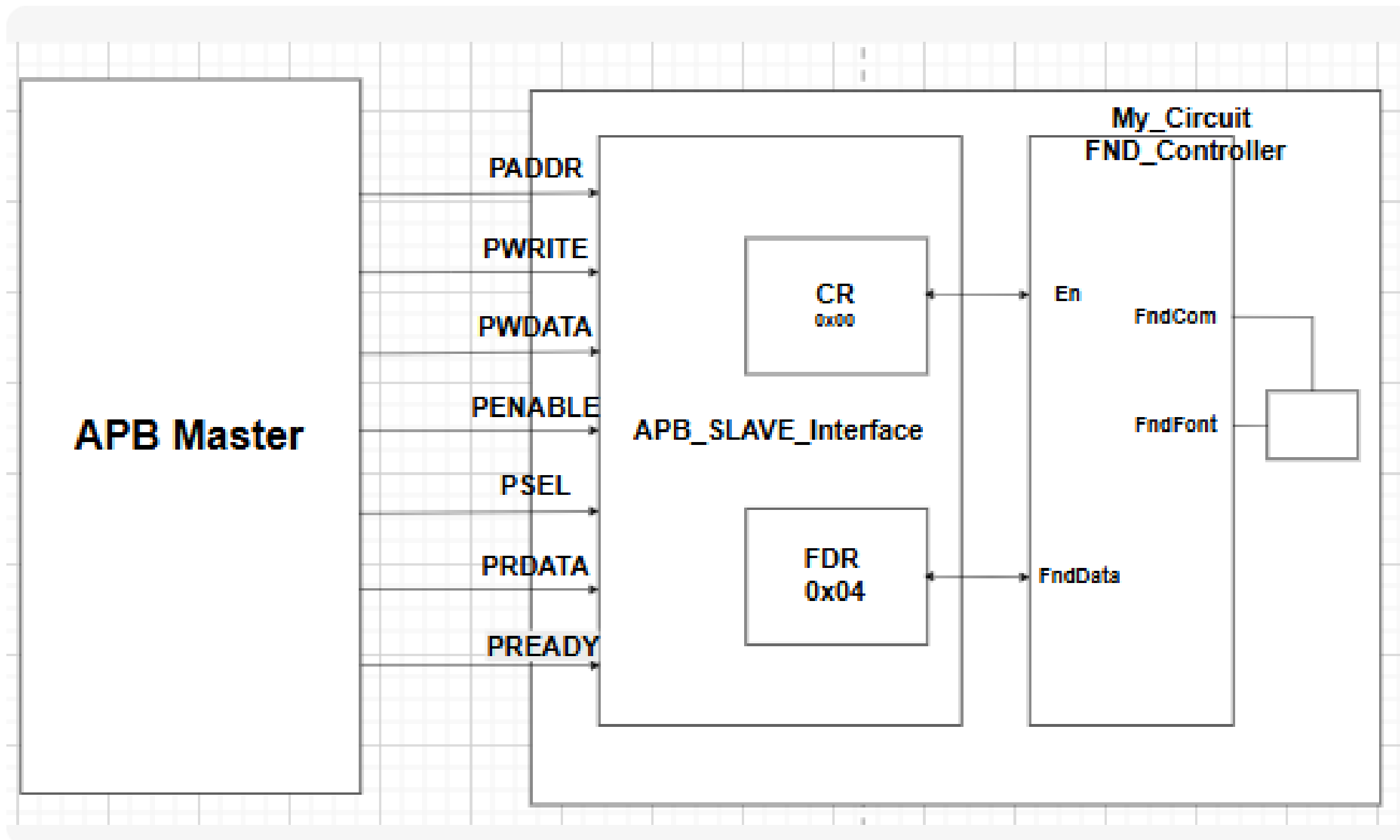
GPIO

Control Register

Input Data Register

Output Data Register

Peripheral

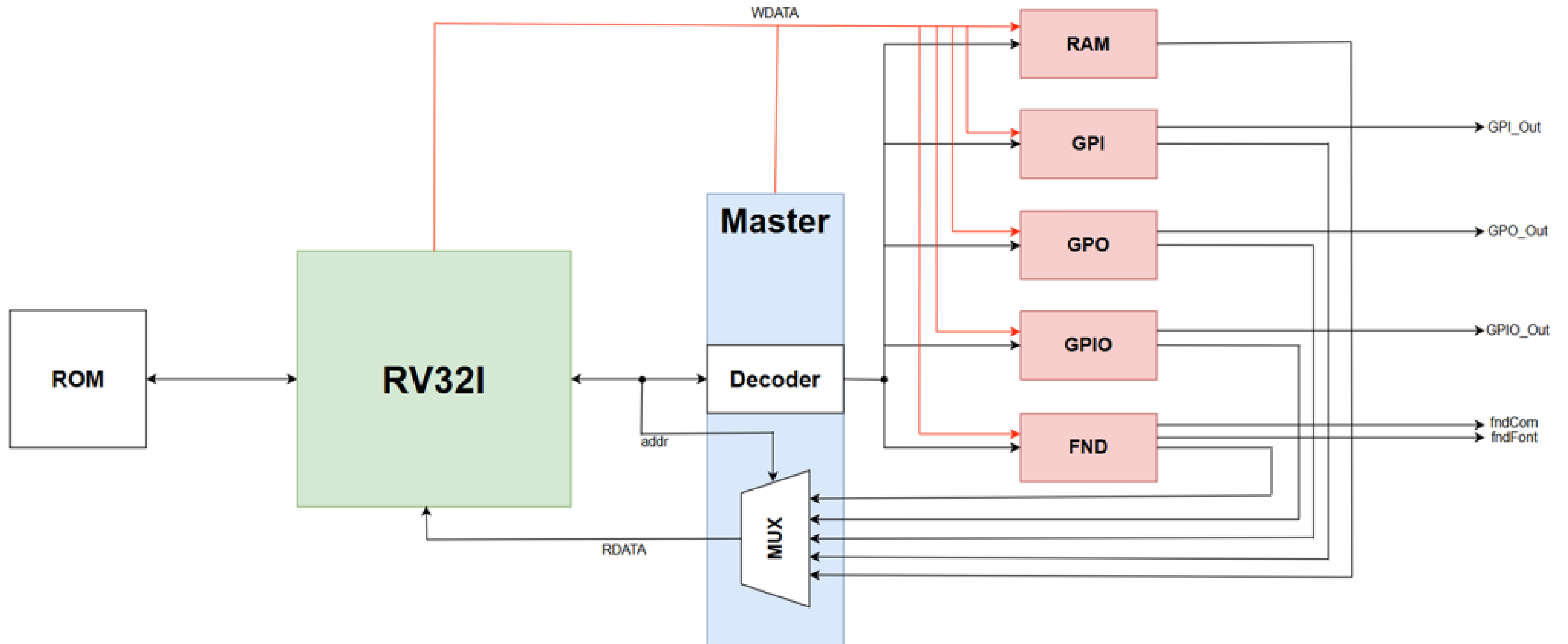


FND_Controller

fndEnable Register

fndData Register

Peripheral

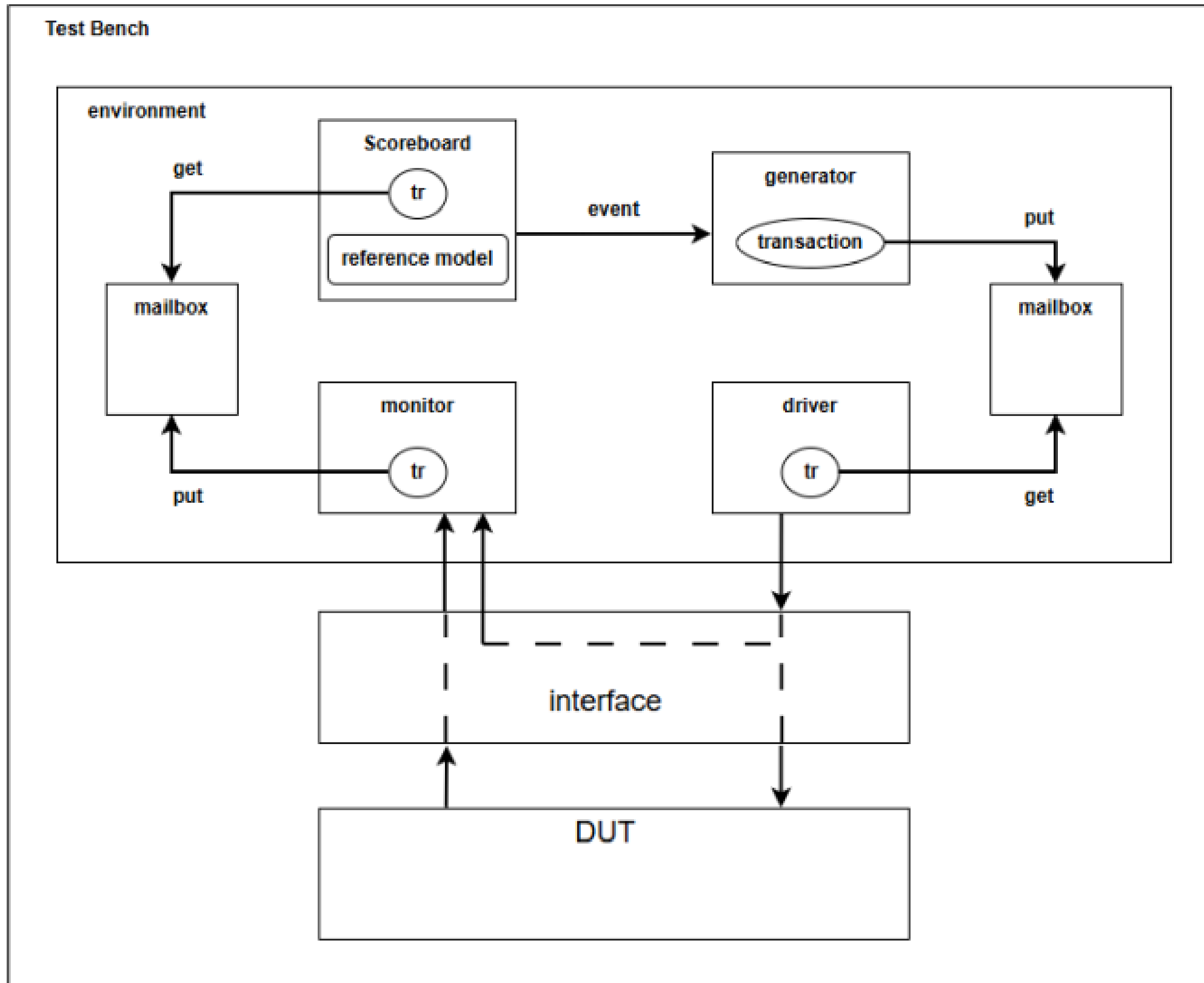




04

Simulation & Verification

Verification

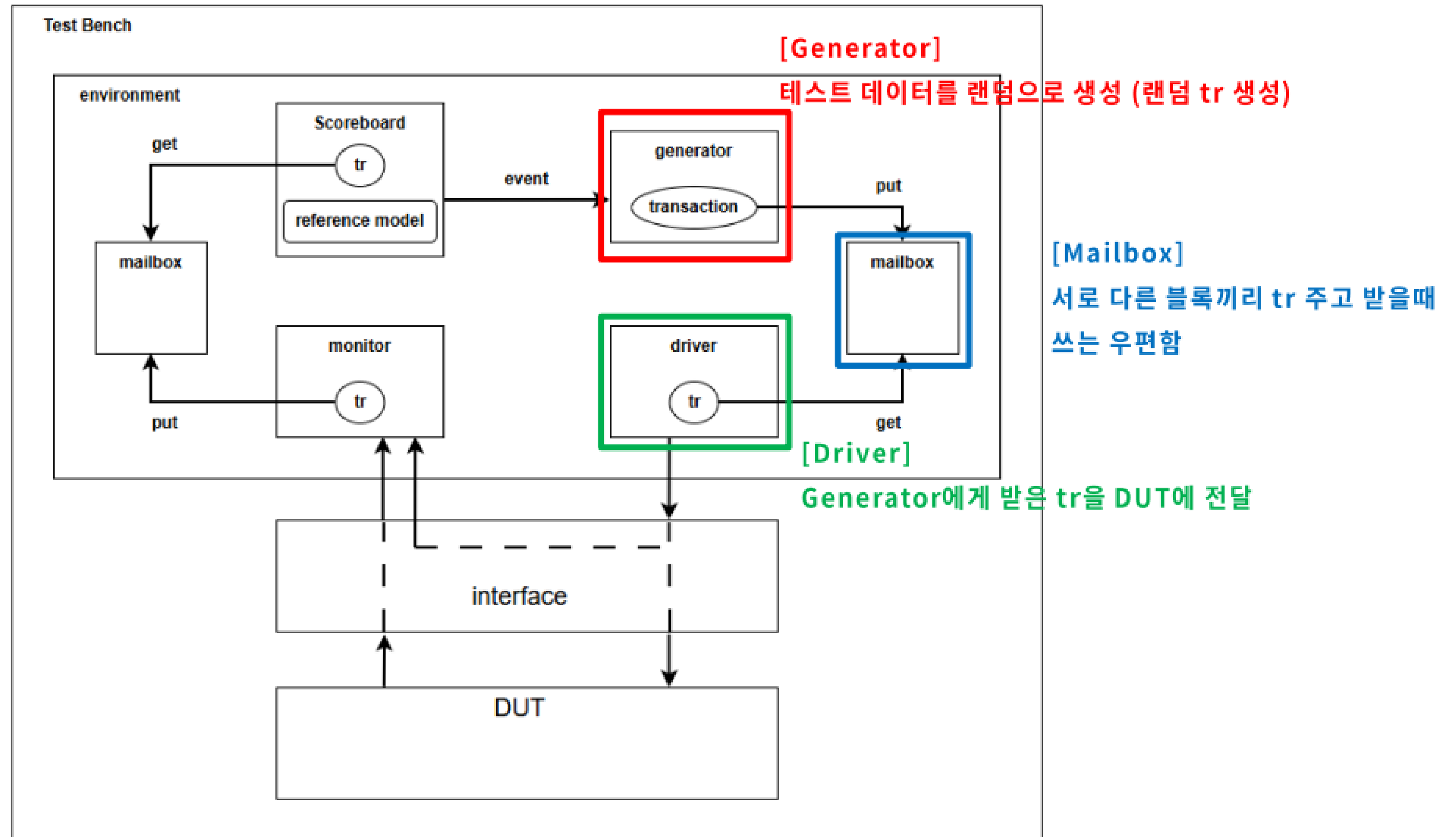


Class 기반 테스트벤치 구조 장점

1. 재사용성과 확장성
2. 객체 지향 프로그래밍 장점 활용(캡슐화, 상속)
3. 독립적인 객체 동작

→ 병렬동작으로 인해 시뮬레이션 시간 단축

Verification



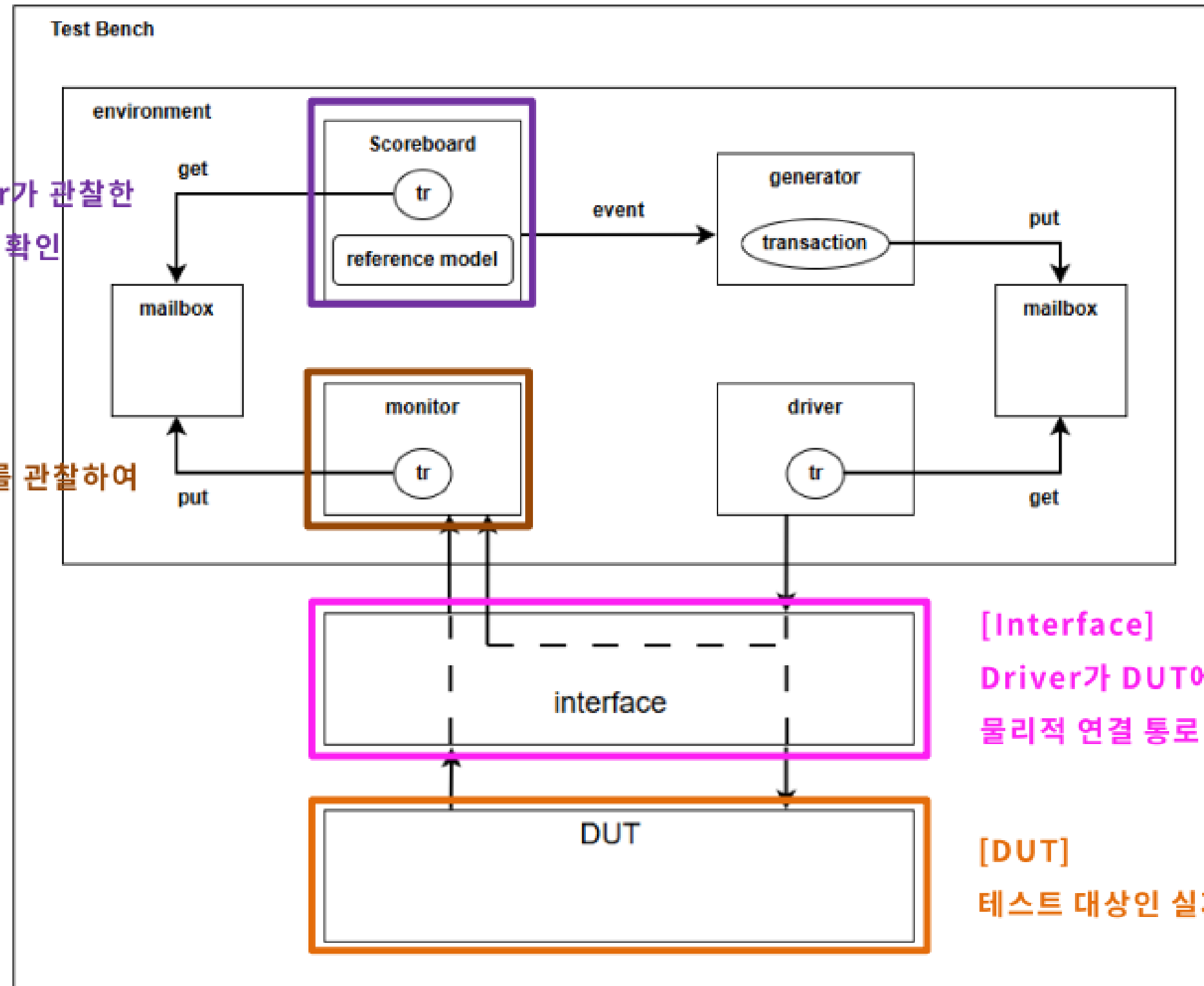
Verification

[Scoreboard]

Generator가 만든 tr과 Monitor가 관찰한 tr을 비교하여 DUT 동작이 맞는지 확인

[Monitor]

DUT에서 실제 발생한 tr를 관찰하여 mailbox로 전달



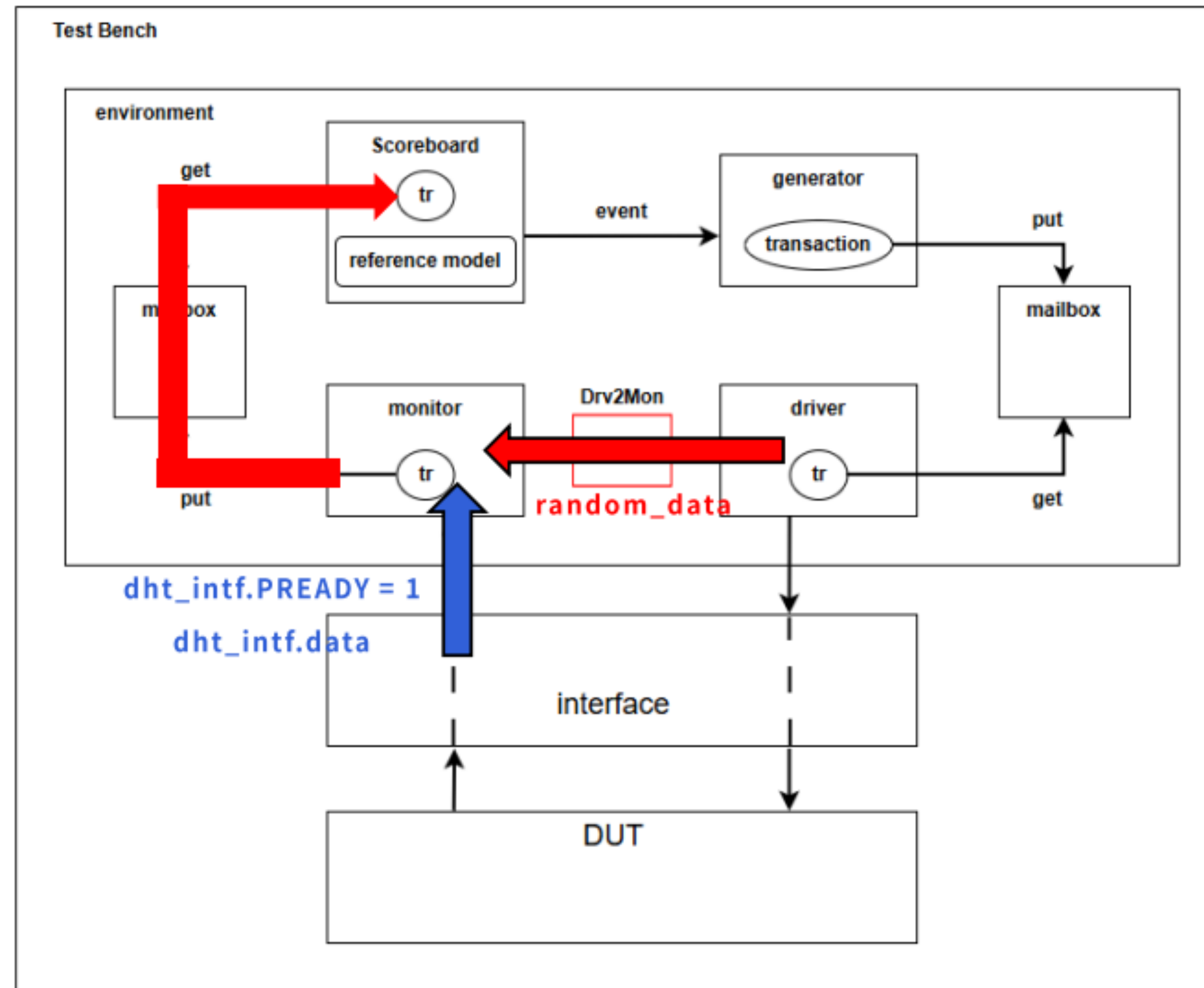
[Interface]

Driver가 DUT에 tr을 전달할 때 사용하는 물리적 연결 경로

[DUT]

테스트 대상인 실제 회로

Verification



Simulation - GPO

```
1  #include <stdint.h>
2
3  #define APB_BASE    0x10000000
4  #define GPO_BASE    (APB_BASE + 0x1000)
5  #define GPO_CR      *(uint32_t *) (GPO_BASE + 0x00)
6  #define GPO_ODR      *(uint32_t *) (GPO_BASE + 0x04)
7
8  int main()
9  {
10     uint32_t data = 1;
11
12     GPO_CR = 0xff;
13     while(1)
14     {
15         GPO_ODR = data;
16         data = (data >> 7) | (data << 1);
17     }
18     return 0;
19 }
```

01

APB 및 GPO의 BASE를 설정합니다.

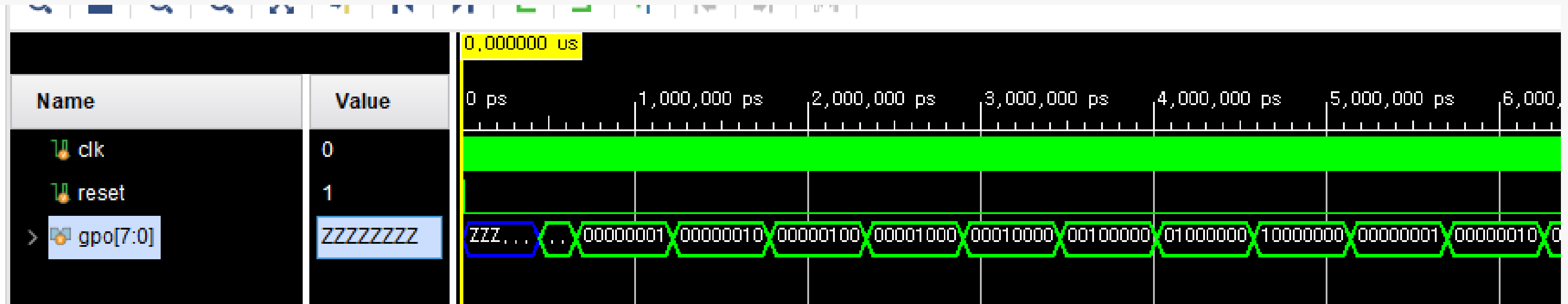
02

CR를 0xff로 설정하여 8개 핀을 출력 모드로 설정합니다

03

Data 값이 비트를 1씩 좌로 이동시키며 GPO핀에 출력

Simulation - GPO



Simulation

1이 좌로 Shift되며 돌아가며 8bit 순환 Shift 연산을 확인 가능

Simulation - GPI



Simulation

입력한 값이 그대로 출력되는 모습을 확인할 수 있다.

Simulation – GPIO

```
1 #include <stdint.h>
2
3 #define APB_BASE 0x10000000
4
5 #define GPO_BASE (APB_BASE + 0x1000)
6 #define GPO_CR *(uint32_t*)(GPO_BASE + 0x00)
7 #define GPO_ODR *(uint32_t*)(GPO_BASE + 0x04)
8
9 #define GPI_BASE (APB_BASE + 0x2000)
10 #define GPI_CR *(uint32_t*)(GPI_BASE + 0x00)
11 #define GPI_IDR *(uint32_t*)(GPI_BASE + 0x04)
12
13 #define GPIO_BASE (APB_BASE + 0x3000)
14 #define GPIO_CR *(uint32_t*)(GPIO_BASE + 0x00)
15 #define GPIO_IDR *(uint32_t*)(GPIO_BASE + 0x04)
16 #define GPIO_ODR *(uint32_t*)(GPIO_BASE + 0x08)
17
18 void delay(uint32_t t);
```

```
int main()
{
    enum {LEFT, RIGHT};

    uint32_t data = 1;

    GPO_CR = 0xff;
    GPI_CR = 0xff;

    GPIO_CR |= 0x0f;
    GPIO_CR &= ~(0x0f<<4);

    uint32_t state = LEFT;

    while(1)
    {
        switch(state)
        {
            case LEFT:
                data = (data >> 7) | (data << 1);
                if (GPIO_IDR & (1<<5)) state = RIGHT;
                break;
            case RIGHT:
                data = (data << 7) | (data >> 1);
                if (GPIO_IDR & (1<<6)) state = LEFT;
                break;
        }
        GPO_ODR = data;
        GPIO_ODR = GPIO_IDR >> 4;
        delay(1000);
    }
    return 0;
}
```

01

APB 및 GPIO의 BASE를 설정합니다.

02

CR를 0xff로 설정하여 8개 핀을 출력 모드로 설정합니다

03

SW를 이용해 LED를 좌우로 이동하도록 설정

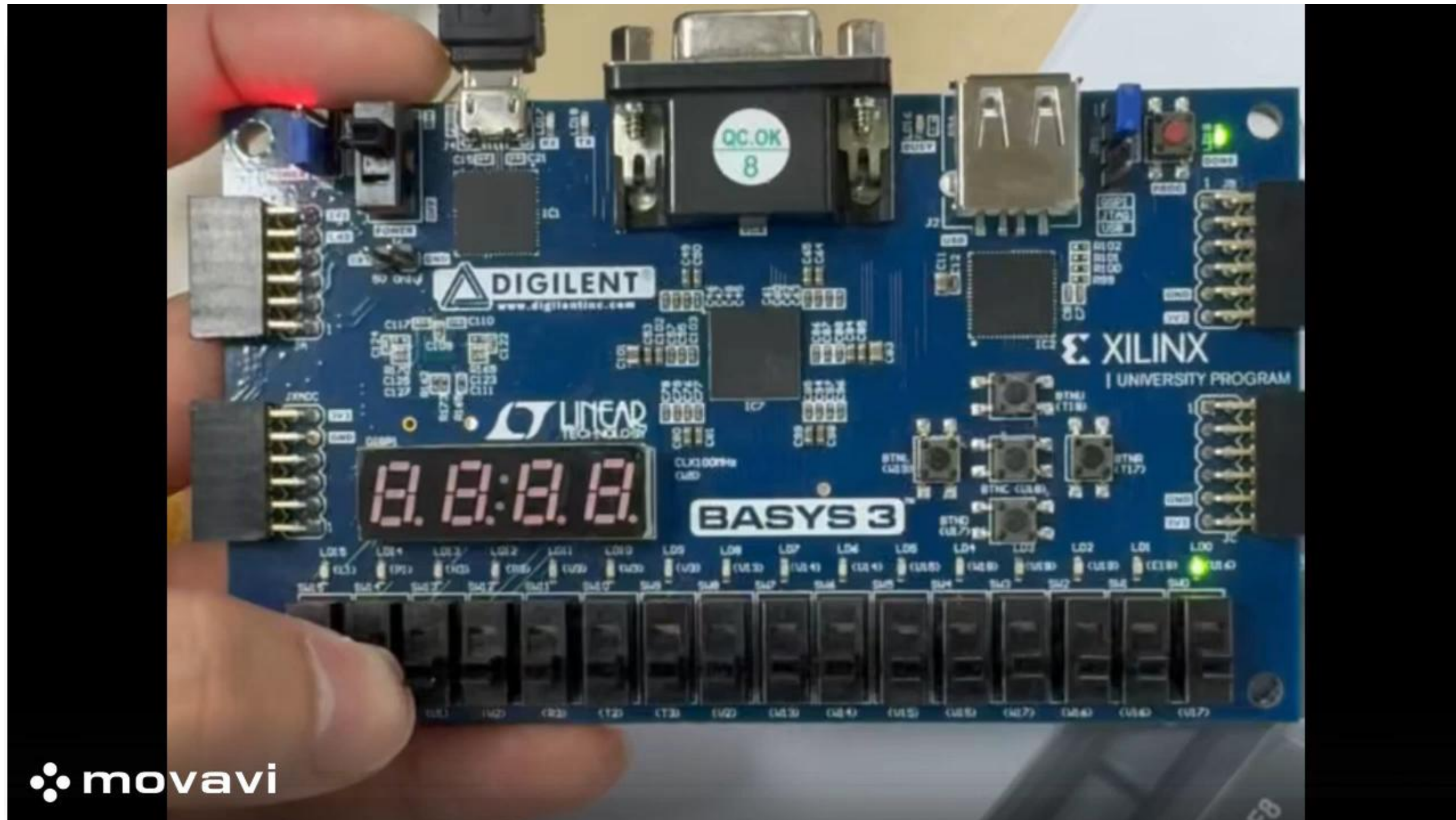
Simulation - GPIO



Simulation

순차적으로 값이 shift 되는 것을 확인할 수 있다.

동작 영상 - GPIO



Simulation - RAM

```
interface ram_intf (  
    input bit clk  
); // clk를 매개변수로 넣을  
    logic [7:0] addr;  
    logic we;  
    logic [7:0] wdata;  
    logic [7:0] rdata;  
endinterface //ram_intf
```

```
class transaction;  
    rand logic [7:0] addr;  
    rand logic      we;  
    rand logic [7:0] wdata;  
    logic      [7:0] rdata;  
  
    task print(string name); // Method  
        $display("[%s] we=%d, addr=%h, wdata=%h, rdata=%h", name, we, addr,  
            wdata, rdata);  
    endtask  
endclass //transaction
```

Simulation - RAM

```

class generator;
  transaction tr;
  mailbox #(transaction) gen2drv_mbox; // Handler 생성 -> 멤버변수
  event scb2gen_event;

  function new(
    mailbox#(transaction) gen2drv_mbox, event scb2gen_event
  ); // 초기화할 때, mailbox 내부의 값을 주겠다. -> 매개변수
    // 만약 this가 없이 이름이 똑같으면? -> 매개변수가 우선순위가
    this.gen2drv_mbox = gen2drv_mbox; // 주소 값을 멤버변수 인
    this.scb2gen_event = scb2gen_event;
  endfunction

  task run(int loop);
    repeat (loop) begin
      tr = new(); // 인스턴스 만들어 연결
      if (!tr.randomize()) $error("Randomization Failed!");
      tr.print("GEN");
      gen2drv_mbox.put(tr); // 포인터를 put해라(넣어라)
      // #20;
      @(scb2gen_event);
    end
  endtask
endclass //generator

```

```

class driver;
  transaction tr; // Handler
  mailbox #(transaction) gen2drv_mbox; // mailbox 정보
  virtual ram_intf ram_if; // 하드웨어인 interfact 연결하기 위함
  event drv2mon_event;

  function new(mailbox#(transaction) gen2drv_mbox, virtual ram_intf ram_if,
    event drv2mon_event);
    this.gen2drv_mbox = gen2drv_mbox;
    this.ram_if = ram_if;
    this.drv2mon_event = drv2mon_event;
  endfunction

  task run();
    forever begin
      gen2drv_mbox.get(tr); // mailbox에서 가져온 값과
      ram_if.addr = tr.addr; // tr의 값을 인터페이스에 넣어준다
      ram_if.we = tr.we;
      if (tr.we)
        ram_if.wdata = tr.wdata; // we가 1일 때, write 해준다.
      //->drv2mon_event; // Event Trigger
      tr.print("DRV");
      @(posedge ram_if.clk); // clk이 발생하면? RAM에 값을 넣어준다
      #1;
    end
  endtask
endclass //driver

```


Simulation - RAM

```

class monitor;
    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    virtual ram_intf ram_if;
    event drv2mon_event;

    function new(mailbox#(transaction) mon2scb_mbox, virtual ram_intf ram_if,
                event drv2mon_event);
        this.mon2scb_mbox = mon2scb_mbox;
        this.ram_if = ram_if;
        this.drv2mon_event = drv2mon_event;
    endfunction

    task run();
        forever begin
            tr = new();
            //@(drv2mon_event);           // Event를 기다림
            @(posedge ram_if.clk);
            #1;
            tr.addr = ram_if.addr; // s/w의 값을 h/w에 넣겠다
            tr.we = ram_if.we;
            tr.wdata = ram_if.wdata;
            if (!tr.we) tr.rdata = ram_if.rdata; // we이 0이어야 읽겠다!
            tr.print("MON");
            mon2scb_mbox.put(tr);
        end
    endtask
endclass //monitor

```

```

class scoreboard;
    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    event scb2gen_event;
    int total_cnt, pass_cnt, fail_cnt;

    logic [7:0] ref_mem[0:2**8-1];
    logic [7:0] ref_rdata;

    function new(mailbox#(transaction) mon2scb_mbox, event scb2gen_event);
        this.mon2scb_mbox = mon2scb_mbox;
        this.scb2gen_event = scb2gen_event;
        this.total_cnt = 0; // 매개변수랑 같은 이름은 없어서 this 없어도 된다
        this.pass_cnt = 0;
        this.fail_cnt = 0;
    endfunction

    task run();
        forever begin
            mon2scb_mbox.get(tr);
            total_cnt++;
            tr.print("SCB");
            if (tr.we) begin
                ref_mem[tr.addr] = tr.wdata;
            end else begin
                ref_rdata = ref_mem[tr.addr];
                if (ref_rdata === tr.rdata) begin
                    pass_cnt++;
                    $display(
                        "PASS! Matched Data! ref_rdata: %h == tr.rdata: %h",
                        ref_rdata, tr.rdata);
                end else begin
                    fail_cnt++;
                    $display(
                        "FAIL! Dismatched Data! ref_rdata: %h != tr.rdata: %h",
                        ref_rdata, tr.rdata);
                end
            end
            $display(""); // 줄 바꿈
            ->scb2gen_event; // Event Trigger
        end
    endtask
endclass //scoreboard

```


Simulation - RAM

```

class environment;
    mailbox #(transaction) gen2drv_mbox;
    mailbox #(transaction) mon2scb_mbox;
    generator              gen;
    driver                  drv;
    monitor                 mon;
    scoreboard              scb;

    event                   drv2mon_event;
    event                   scb2gen_event;

    function new(virtual ram_intf ram_if);
        gen2drv_mbox = new(); // 인스턴스 생성
        mon2scb_mbox = new();
        gen           = new(gen2drv_mbox, scb2gen_event);
        drv           = new(gen2drv_mbox, ram_if, drv2mon_event);
        mon           = new(mon2scb_mbox, ram_if, drv2mon_event);
        scb           = new(mon2scb_mbox, scb2gen_event);
    endfunction //new()

    task run(int loop);
        fork
            gen.run(loop);
            drv.run();
            mon.run();
            scb.run();
        join_any
        $display("Total : %d", scb.total_cnt);
        $display("Pass : %d", scb.pass_cnt);
        $display("Fail : %d", scb.fail_cnt);
        #50;
    endtask
endclass //environment

```

```

module tb_ram ();
    bit clk;
    ram_intf ram_if (clk);
    environment env;

    ram DUT (
        .clk(clk),
        .addr(ram_if.addr),
        .we(ram_if.we),
        .wdata(ram_if.wdata),
        .rdata(ram_if.rdata)
    );

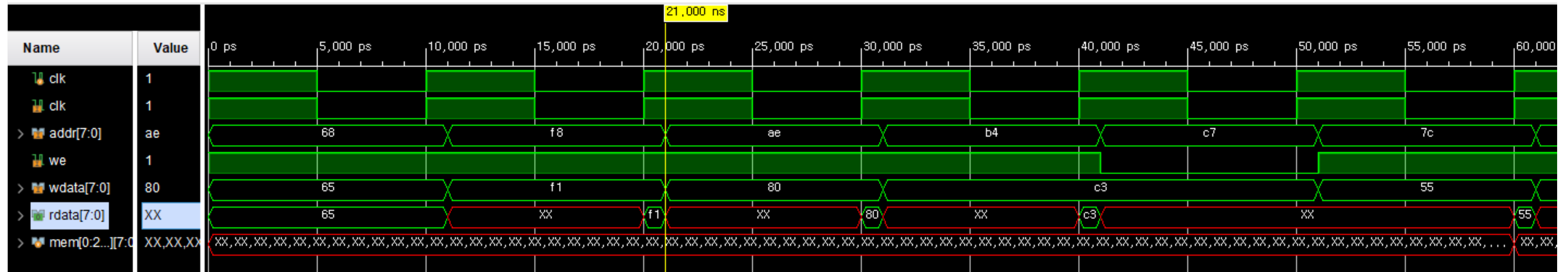
    always #5 clk = ~clk;

    initial begin
        clk = 1;
    end

    initial begin
        env = new(ram_if);
        env.run(1000);
        $finish;
    end
endmodule

```

Simulation - RAM



Time resolution is 1 ps

```
[GEN] we=1, addr=68, wdata=65, rdata=xx
[DRV] we=1, addr=68, wdata=65, rdata=xx
[MON] we=1, addr=68, wdata=65, rdata=xx
[SCB] we=1, addr=68, wdata=65, rdata=xx
```

```
[GEN] we=1, addr=f8, wdata=f1, rdata=xx
[DRV] we=1, addr=f8, wdata=f1, rdata=xx
[MON] we=1, addr=f8, wdata=f1, rdata=xx
[SCB] we=1, addr=f8, wdata=f1, rdata=xx
```

```
[GEN] we=1, addr=ae, wdata=80, rdata=xx
[DRV] we=1, addr=ae, wdata=80, rdata=xx
[MON] we=1, addr=ae, wdata=80, rdata=xx
[SCB] we=1, addr=ae, wdata=80, rdata=xx
```

```
[GEN] we=1, addr=b4, wdata=c3, rdata=xx
[DRV] we=1, addr=b4, wdata=c3, rdata=xx
[MON] we=1, addr=b4, wdata=c3, rdata=xx
[SCB] we=1, addr=b4, wdata=c3, rdata=xx
```

```
[GEN] we=0, addr=c7, wdata=7f, rdata=xx
[DRV] we=0, addr=c7, wdata=7f, rdata=xx
[MON] we=0, addr=c7, wdata=c3, rdata=xx
[SCB] we=0, addr=c7, wdata=c3, rdata=xx
PASS! Matched Data! ref_rdata: xx == tr.rdata: xx
```

```
[GEN] we=1, addr=7c, wdata=55, rdata=xx
[DRV] we=1, addr=7c, wdata=55, rdata=xx
[MON] we=1, addr=7c, wdata=55, rdata=xx
[SCB] we=1, addr=7c, wdata=55, rdata=xx
```

```
[GEN] we=1, addr=a6, wdata=df, rdata=xx
[DRV] we=1, addr=a6, wdata=df, rdata=xx
[MON] we=1, addr=a6, wdata=df, rdata=xx
[SCB] we=1, addr=a6, wdata=df, rdata=xx
```

```
[GEN] we=0, addr=6d, wdata=06, rdata=xx
[DRV] we=0, addr=6d, wdata=06, rdata=xx
[MON] we=0, addr=6d, wdata=df, rdata=xx
[SCB] we=0, addr=6d, wdata=df, rdata=xx
PASS! Matched Data! ref_rdata: xx == tr.rdata: xx
```

```
[GEN] we=1, addr=db, wdata=25, rdata=xx
[DRV] we=1, addr=db, wdata=25, rdata=xx
[MON] we=1, addr=db, wdata=25, rdata=xx
[SCB] we=1, addr=db, wdata=25, rdata=xx
```

```
[GEN] we=0, addr=fc, wdata=77, rdata=xx
[DRV] we=0, addr=fc, wdata=77, rdata=xx
[MON] we=0, addr=fc, wdata=25, rdata=xx
[SCB] we=0, addr=fc, wdata=25, rdata=xx
```

PASS! Matched Data! ref_rdata: xx == tr.rdata: xx

```
Total :      10
Pass :      3
Fail :      0
```

Simulation - FND

```
#define FND_BASE (APB_BASE + 0x4000)
#define FND_DATA *(uint32_t*)(FND_BASE + 0x00)

void delay(uint32_t t);

int main()
{
    uint32_t data = 0;

    while(1)
    {
        if (data == 9999) data = 0;
        else data += 1;

        FND_DATA = data;
        delay(1000);
    }

    return 0;
}
```

01

APB 및 FND의 BASE를 설정합니다.

02

Data의 위치를 설정해줍니다

03

0~9999까지 세는 FND 출력을 보여줍니다.

Simulation - FND

UP COUNTER

C code

```
typedef struct{
    uint32_t CR;
    uint32_t FDR;
} FND_TypeDef;

#define APB_BASE 0x10000000
#define FND_BASE (APB_BASE + 0x4000)
#define FND ((FND_TypeDef*)(FND_BASE))

void delay(uint32_t t);

int main()
{
    //FND_CR = 0x01;
    //FND_FDR = 1234;

    FND->CR = 0x01;
    FND->FDR = 1234;

    uint32_t data = 0;

    while (1)
    {
        FND->FDR = data;
        data++;
        delay(1000);
    }

    return 0;
}
```



ASM

```
main:
    addi    sp,sp,-32
    sw      ra,28(sp)
    sw      s0,24(sp)
    addi    s0,sp,32
    li      a5,268451840
    li      a4,1
    sw      a4,0(a5)
    li      a5,268451840
    li      a4,1234
    sw      a4,4(a5)
    sw      zero,-20(s0)

.L2:
    li      a5,268451840
    lw      a4,-20(s0)
    sw      a4,4(a5)
    lw      a5,-20(s0)
    addi    a5,a5,1
    sw      a5,-20(s0)
    li      a0,1000
    call    delay
    j       .L2

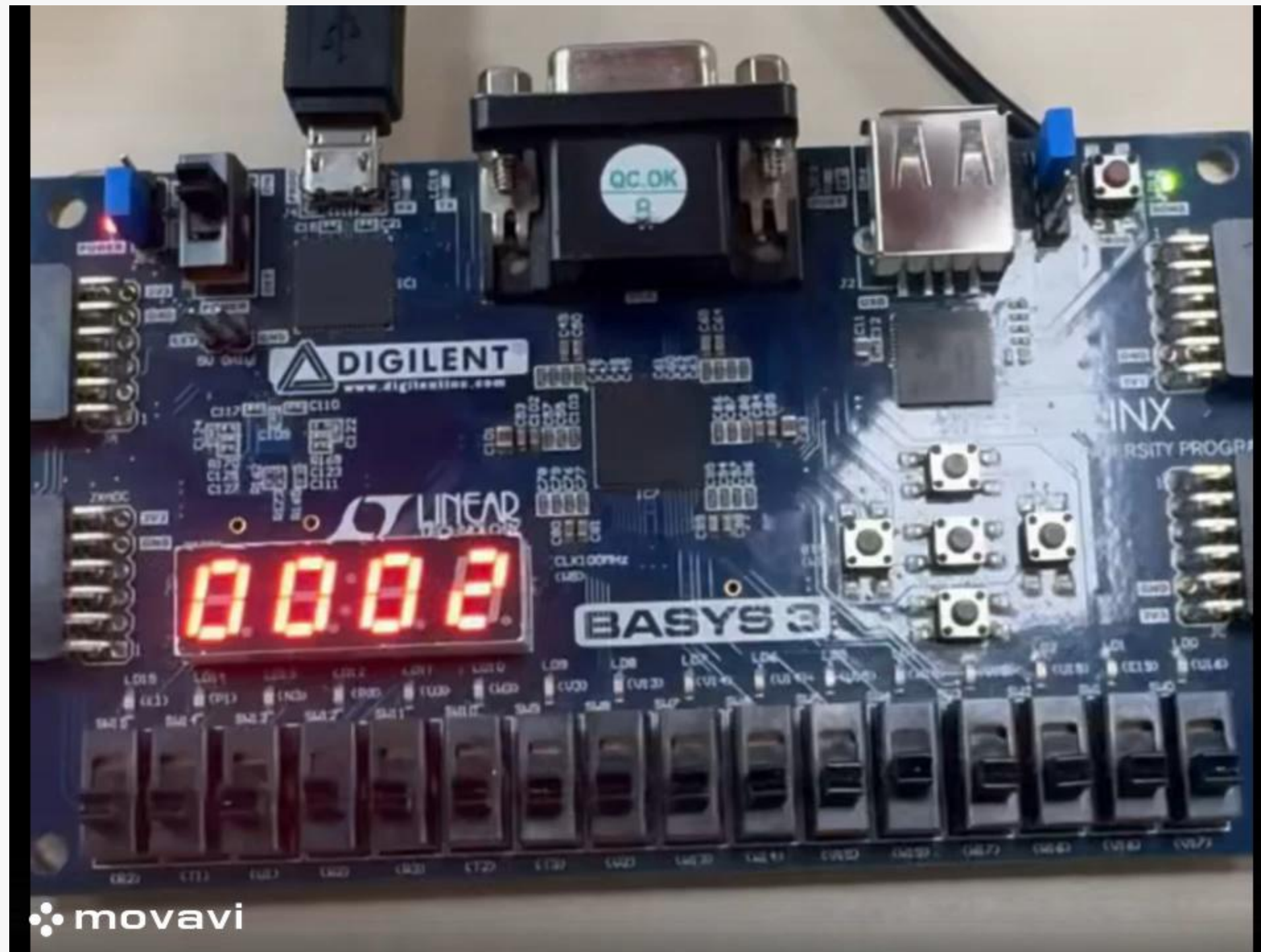
delay:
    addi    sp,sp,-48
    sw      ra,44(sp)
    sw      s0,40(sp)
    addi    s0,sp,48
```



Machine Code

```
2  fe010113
3  00112e23
4  00812c23
5  02010413
6  10004537
7  02c000ef
8  fe042623
9  fec42583
10 10004537
11 050000ef
12 fec42783
13 00178793
14 fef42623
15 3e800513
16 074000ef
17 fe1ff06f
```


동작 영상 – FND





05

Troubleshooting

Troubleshooting

```
module APB_Decoder (  
    input logic      en,  
    input logic [31:0] sel,  
    output logic [ 4:0] y,  
    output logic [ 2:0] mux_sel  
);  
  
    always_comb begin  
        y = 5'b00000;  
        if (en) begin  
            casex (sel)  
                32'h1000_0xxx: y = 5'b00001;  
                32'h1000_1xxx: y = 5'b00010;  
                32'h1000_2xxx: y = 5'b00100;  
                32'h1000_3xxx: y = 5'b01000;  
                32'h1000_4xxx: y = 5'b10000;  
            endcase  
        end  
    end  
end
```

MASTER → Mux_Sel Signal

기존 2bit에서 FND를 추가하며 신호가
늘어났지만 적용하지 않음

따라서 메모리 주소가 할당되지 않던 상황을
확인 후 수정



06

고찰

고찰



01

메모리 맵핑에 대한 이해

단순 주소 할당 작업이 아닌, C코드를 작성하며 Register 주소 기준으로 Peripheral을 제어하는 것을 체감

02

검증 환경의 이해

테스트벤치를 설계하며 mailbox를 이용한 정보 전달 구조를 사용하며 Class기반 계층적 검증 흐름을 습득

03

검증의 핵심

단순 시뮬 결과가 아닌 DUT의 움직임을 알고 타이밍 맞추기

감사합니다